ФГАОУ ВО «Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)» (МГТУ им. Н.Э. Баумана)
ИУ-5 «Системы обработки информации и управления»

Лабораторная работа № 1 (часть 2) «Создание базы данных в DBeaver и обеспечение целостности таблиц»

Цель: изучить процесс создания новой базы данных SQLite в среде DBeaver, освоить работу с графическим конструктором таблиц и с SQL-редактором для проектирования структуры базы. Изучить способы обеспечения целостности данных в реляционных таблицах SQLite с использованием ограничений. Сформировать практические навыки добавления и настройки ограничений NOT NULL, UNIQUE, CHECK, DEFAULT, а также внешних ключей (FOREIGN KEY) и каскадных операций (ON DELETE CASCADE, ON UPDATE CASCADE) с помощью графического интерфейса DBeaver и SQL-запросов. Научиться проверять корректность работы ограничений на тестовых данных и анализировать возникающие ошибки.

Порядок выполнения лабораторной работы

1. Создать новую базу данных SQLite через интерфейс DBeaver.
2. Через визуальный конструктор DBeaver создать часть таблиц вашей предметной области.
3. С помощью SQL-редактора DBeaver создать оставшиеся таблицы.
4. Сравнить результаты обоих подходов.
5. Сохранить (экспортировать) SQL-скрипт структуры базы.
6. Создать резервную копию базы данных.
7. Построить ER-диаграмму через интерфейс DBeaver.
8. Сравнить с ER-диаграммой, полученной в предыдущей лабораторной работе.
9. Убедиться, что все таблицы созданы.
10. Проанализировать структуру базы данных и определить связанные таблицы, между которыми должна быть обеспечена ссылочная целостность.
11. Добавить ограничения **FOREIGN KEY** между всеми логически связанными таблицами базы данных.
12. Для ключевых и обязательных атрибутов таблиц добавить ограничения **NOT NULL.**
13. Добавить ограничения **UNIQUE** для атрибутов, значения которых не должны повторяться в рамках таблицы.

14. Настроить значения по умолчанию (**DEFAULT**) для отдельных атрибутов таблиц.
15. Создать ограничения **CHECK** для реализации бизнес-правил предметной области (ограничения на диапазоны значений, допустимые состояния и т.п.).
16. Настроить каскадные операции (**ON DELETE CASCADE, ON UPDATE CASCADE**) для внешних ключей.
17. Проверить работу всех добавленных ограничений на тестовых данных:

- Попытка вставки некорректных данных;
- Попытка удаления и обновления связанных записей;
- Анализ возникающих ошибок и их причин.

Теоретическая часть Установка и настройка SQLite и DBeaver

DBeaver – бесплатная программа для работы с различными СУБД, позволяющая создавать новые базы данных и управлять ими. Для работы с SQLite в DBeaver отдельный сервер не нужен – база просто хранится в файле. После установки DBeaver (требуется Java) следует создать соединение: в мастере подключения выберите тип SQLite, укажите путь к файлу базы (можно создать новый). Например, в диалоге создания соединения (см. Рисунок 1) вы указываете файл *.sqlite; если он отсутствует, DBeaver создаст новую пустую базу. Далее нажмите «Тестировать подключение» – при успехе соединение будет установлено.

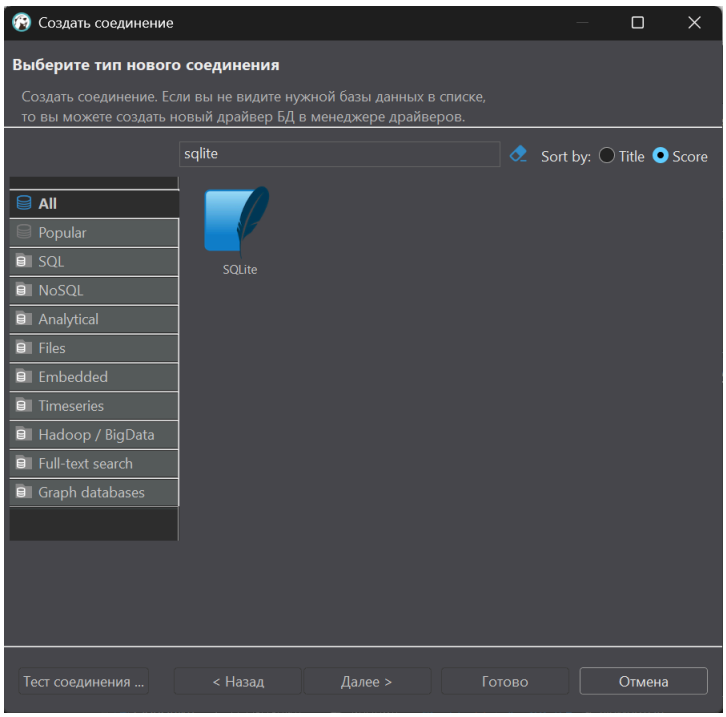


Рисунок 1 – Окно создания нового соединения к SQLite в DBeaver

Основные элементы базы данных и таблицы

База данных SQLite – это один файл, содержащий набор таблиц. Каждая таблица имеет имя и набор столбцов (полей), описывающих структуру записей. При создании таблицы с помощью **CREATE TABLE** указываются имя таблицы и перечень её полей (каждое поле имеет имя и тип данных). Например, название таблицы и список полей описывается так:

```
CREATE TABLE departments (
   id   INTEGER,
   name TEXT
);
```

Здесь **departments** – имя таблицы, а id и name – имена столбцов с типами **INTEGER** и **TEXT**. После создания таблицы она появится в дереве соединения DBeaver в навигаторе (Database Navigator).

Типы данных SQLite

В SQLite применяется динамическая система типов. Для столбцов доступны пять основных классов хранения (типов данных):

1. **NULL** – отсутствие значения (пустое значение);

2. **INTEGER** – целое число (может занимать 1, 2, 3, 4, 6 или 8 байт в зависимости от величины);
3. **REAL** – число с плавающей точкой (обычно 8 байт);
4. **TEXT** – строка текста (в кавычках, в кодировке БД);
5. **BLOB** – бинарные данные (произвольная последовательность байт).

SQLite хранит тип данных вместе со значением, то есть тип привязан к данным, а не фиксирован на столбце. Несмотря на этот гибкий подход, при объявлении таблицы мы обычно указываем один из пяти названных типов. Например, можно объявить **age NUMERIC**, но фактически данные этого столбца будут храниться в одном из базовых классов хранения (**INTEGER**, **REAL** и т.д.).

Синтаксис CREATE TABLE

Команда CREATE TABLE используется для создания новой таблицы в базе данных. Она должна содержать имя создаваемой таблицы и описание всех её столбцов. Синтаксис таков:

1. Имя таблицы.
2. Перечень столбцов: для каждого указывают имя столбца и его тип (и, при необходимости, ограничения).
3. Опционально: объявление первичного ключа или других ограничений.

Например:

```
CREATE TABLE employees (
   id          INTEGER,
   name        TEXT,
   department_id INTEGER
);
```

Здесь **employees** – имя таблицы, а у неё три столбца с типами **INTEGER** и **TEXT**. Если всё корректно, DBeaver выполнит команду и создаст таблицу.

Первичные ключи (PRIMARY KEY)

У каждой таблицы обычно задаётся первичный ключ – столбец (или комбинация столбцов), однозначно идентифицирующих строку. Атрибут **PRIMARY KEY** обеспечивает уникальность и быстрое обращение к записям. Например:

```
CREATE TABLE employees (
   id   INTEGER PRIMARY KEY,
   name TEXT
```

В этом примере столбец **id** является первичным ключом – его значения не могут повторяться. После создания таблицы ни одна вторая запись не сможет иметь то же значение **id**. Первичный ключ можно указывать прямо при описании столбца (как в примере) или в конце списка полей, например **PRIMARY KEY(id)**.

Дополнительно можно использовать AUTOINCREMENT для целочисленного ключа: тогда при вставке новых записей значение id будет автоматически увеличиваться на 1. Например:

```
CREATE TABLE departments (
   id   INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT
);
```

Если ключ составной (несколько полей), его объявляют как **PRIMARY KEY(col1, col2)** после описания полей.

Интерфейс DBeaver

DBeaver предоставляет единый графический интерфейс для разных СУБД. Слева находится Database Navigator – панель, где видны созданные подключения и объекты баз (таблицы, индексы и т.д.). После установки соединения с вашей новой базой SQLite в навигаторе появится узел с её именем.

Через контекстное меню на соединении (правый клик) можно создавать таблицы графически: выберите «Создать новую таблицу». Откроется диалог конструктора таблицы, где можно добавить поля и выбрать их типы. Этот визуальный редактор особенно удобен, если не хочется сразу писать SQL.

Чтобы работать через SQL, в DBeaver выполните правый клик на соединении и выберите «Редактор SQL → Новый SQL-скрипт». В появившемся окне редактора можно вводить любые SQL-команды (например, **CREATE TABLE** ... для создания таблиц). Так в DBeaver легко комбинировать подходы: часть таблиц создать через GUI, часть – через SQL.

SQL-скрипты и создание таблиц

SQL-скрипт – это набор SQL-команд, сохраняемых и выполняемых вместе. В DBeaver, после открытия SQL-редактора, вы можете сразу писать несколько операторов **CREATE TABLE** подряд. Например:

```
CREATE TABLE departments (
   id   INTEGER PRIMARY KEY,
   name TEXT
);
CREATE TABLE employees (
   id          INTEGER PRIMARY KEY,
   name        TEXT,
   department_id INTEGER
);
```

В этом примере сначала создаётся таблица departments, затем – employees. После выполнения скрипта обе таблицы появятся в базе. Работа через скрипты позволяет быстро воспроизводить схему базы или вносить изменения.

Резервное копирование

Резервная копия базы данных нужна, чтобы сохранить её состояние. Для SQLite самый простой способ резервного копирования – это скопировать файл базы данных. Так как вся база хранится в одном файле, достаточно остановить соединение и продублировать файл (например, database.sqlite). Этот способ безопасен и универсален.

В DBeaver также можно использовать средства экспорта: например, «Tools → Backup» (Dump) для других СУБД, либо экспортировать данные и схему в SQL-скрипт через мастер экспорта. Но для учебной работы достаточно скопировать файл *.sqlite из файловой системы – получившийся файл и будет резервной копией базы.

Целостность данных и роль ограничений

Целостность данных — это совокупность правил, обеспечивающих корректность, согласованность и логическую непротиворечивость информации, хранимой в базе данных. Нарушение целостности приводит к появлению ошибочных, противоречивых или бессмысленных данных, что делает невозможной корректную работу приложений и анализ информации.

В реляционных базах данных целостность поддерживается с помощью **ограничений (constraints)**, которые накладываются на таблицы и их столбцы. Ограничения проверяются СУБД автоматически при выполнении операций **INSERT**, **UPDATE** и **DELETE**.

Основные задачи ограничений:

1. Предотвращение ввода некорректных данных;
2. Соблюдение бизнес-правил предметной области;
3. Поддержание связей между таблицами;
4. Снижение количества логических ошибок на уровне приложений.

В SQLite ограничения могут задаваться:

1. При создании таблицы (**CREATE TABLE**);
2. При изменении структуры таблицы (**ALTER TABLE**);
3. Через графический интерфейс инструментов администрирования (например, DBeaver).

Ограничение NOT NULL

Ограничение **NOT NULL** запрещает хранение неопределённых значений (**NULL**) в столбце. Оно используется для атрибутов, которые являются обязательными с точки зрения предметной области.

Если столбец объявлен как **NOT NULL**, каждая запись в таблице должна содержать для него конкретное значение. Попытка вставить или обновить запись с **NULL** приведёт к ошибке.

NOT NULL применяется, например, для:

1. Идентификаторов;
2. Имён, названий, кодов;
3. Дат создания объектов;
4. Ключевых атрибутов, без которых запись теряет смысл.

Пример объявления ограничения **NOT NULL**:

```
CREATE TABLE employers (
   id_employer INTEGER PRIMARY KEY,
   surname TEXT NOT NULL,
   name TEXT NOT NULL
);
```

В этом примере фамилия и имя работника являются обязательными атрибутами.

Ограничение UNIQUE

Ограничение **UNIQUE** обеспечивает уникальность значений в одном или нескольких столбцах таблицы. Оно используется в тех случаях, когда значения не должны повторяться в пределах таблицы.

UNIQUE может применяться:

1. к одному столбцу (уникальный идентификатор, код, email);
2. к группе столбцов (составная уникальность).

Важно отметить, что:

1. уникальность проверяется только для ненулевых значений;
2. ограничение **UNIQUE** не заменяет первичный ключ, но дополняет его.

Пример объявления **UNIQUE**:

```
CREATE TABLE employers (
   id_employer INTEGER PRIMARY KEY,
   email TEXT UNIQUE
);
```

В данном случае каждый работник должен иметь уникальный адрес электронной почты. Пример составного ограничения **UNIQUE**:

```
UNIQUE (surname, name)
```

Такое ограничение запрещает существование двух записей с одинаковым сочетанием фамилии и имени.

Значения по умолчанию (DEFAULT)

Ограничение **DEFAULT** задаёт значение, которое автоматически подставляется в столбец при вставке новой записи, если явное значение не было указано.

DEFAULT используется для:

1. Статусов объектов;
2. Дат создания записей;
3. Логических флагов;
4. Часто повторяющихся значений.

Использование **DEFAULT**:

1. Упрощает операции вставки данных;
2. Снижает вероятность ошибок;
3. Обеспечивает единообразие значений.

Пример задания значения по умолчанию:

```
CREATE TABLE employers (
   id_employer INTEGER PRIMARY KEY,
   status TEXT DEFAULT 'active'
);
```

Если при добавлении записи поле status не указано, оно автоматически примет значение 'active'.

Ограничение CHECK

Ограничение **CHECK** используется для задания логических условий, которым должны удовлетворять значения столбцов. Оно позволяет реализовать бизнес-правила предметной области непосредственно на уровне базы данных.

CHECK может:

1. ограничивать диапазон допустимых значений;
2. проверять логические условия;
3. использовать выражения и логические операторы.

Примеры бизнес-ограничений:

1. числовое значение должно быть положительным;
2. возраст должен находиться в заданном диапазоне;

3. статус может принимать только допустимые значения.

Пример ограничения **CHECK**:

```
CREATE TABLE employers (
   id_employer INTEGER PRIMARY KEY,
   salary REAL CHECK (salary > 0)
);
```

Более сложный пример:

```
CHECK (age >= 18 AND age <= 65)
```

Внешние ключи и ссылочная целостность (FOREIGN KEY)

Ограничение **FOREIGN KEY** используется для установления связей между таблицами и обеспечения ссылочной целостности данных.

Ссылочная целостность означает, что:

1. значение внешнего ключа в дочерней таблице должно существовать в родительской таблице;
2. невозможно создать «висячие» ссылки на несуществующие записи.

Типичная ситуация — связь между справочной таблицей и основной таблицей.

Пример структуры:

1. departments — справочник подразделений;
2. employers — работники, относящиеся к подразделениям.

```
CREATE TABLE departments (
   id_department INTEGER PRIMARY KEY,
   name TEXT NOT NULL
);

CREATE TABLE employers (
   id_employer INTEGER PRIMARY KEY,
   id_department INTEGER,
   FOREIGN KEY (id_department)
      REFERENCES departments(id_department)
);
```

В этом случае каждый работник может быть связан только с существующим подразделением.

Каскадные операции

Каскадные операции определяют поведение дочерних записей при изменении или удалении связанных записей в родительской таблице.

Наиболее распространённые варианты:

1. **ON DELETE CASCADE**
2. **ON UPDATE CASCADE**

Использование каскадных операций позволяет:

1. Автоматически поддерживать целостность данных;
2. Избежать ручного удаления связанных записей;
3. Снизить вероятность логических ошибок.

Пример задания каскадных операций:

```
FOREIGN KEY (id_department)
REFERENCES departments(id_department)
ON DELETE CASCADE
ON UPDATE CASCADE
```

1. **ON DELETE CASCADE** — при удалении записи из родительской таблицы все связанные записи удаляются автоматически.
2. **ON UPDATE CASCADE** — при изменении значения ключа в родительской таблице оно автоматически обновляется во всех связанных таблицах.

Контрольные вопросы

1. Каково назначение СУБД DBeaver и какие СУБД она поддерживает?
2. В чём особенность базы данных SQLite по сравнению с серверными СУБД?
3. Как осуществляется создание новой базы данных SQLite в DBeaver?
4. Что представляет собой база данных SQLite с точки зрения хранения данных?
5. Что такое таблица в базе данных и из каких элементов она состоит?

6. Какие основные типы данных (классы хранения) используются в SQLite?
7. В чём заключается особенность системы типов данных в SQLite?
8. Каково назначение команды CREATE TABLE и каков её общий синтаксис?
9. Что такое первичный ключ (PRIMARY KEY) и для чего он используется?
10. Что означает свойство AUTOINCREMENT и когда его целесообразно применять?
11. Какие способы создания таблиц предоставляет DBeaver и в чём их различие?
12. Что такое SQL-скрипт и для каких задач он используется при проектировании базы данных?
13. Что понимается под целостностью данных в реляционных базах данных? Зачем она необходима?
14. Что такое ограничения (constraints) в SQL и на каких этапах они проверяются СУБД?
15. Какими способами в SQLite можно задать ограничения для таблиц?
16. Каково назначение ограничения NOT NULL? В каких случаях его необходимо использовать?
17. Что обеспечивает ограничение UNIQUE? В чём его отличие от первичного ключа?
18. Что такое составное ограничение UNIQUE и когда оно применяется?
19. Каково назначение ограничения DEFAULT? В каких ситуациях целесообразно использовать значения по умолчанию?
20. Для чего используется ограничение CHECK? Какие бизнес-правила можно реализовать с его помощью?
21. Что такое внешний ключ (FOREIGN KEY) и какую роль он играет в обеспечении ссылочной целостности?
22. Какие требования накладываются на данные при использовании внешних ключей? Что такое «висячие ссылки»?
23. Что такое каскадные операции? В чём назначение ON DELETE CASCADE и ON UPDATE CASCADE?
24. Какие действия с данными могут привести к нарушению ограничений и какие ошибки при этом возникают?
25. Почему рекомендуется реализовывать правила целостности на уровне базы данных, а не только на уровне приложения?